

P1304R0

Simplifying Extern Template

Published Proposal, 2018-10-07

This version:

<https://wg21.link/p1304r0>

Author:

[Isabella Muerte](#)

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Render:

[P1304](#)

Current Source:

[slurps-mad-rips/papers/proposals/simplify-extern-template.bs](https://ericniebler.com/2018/10/07/simplify-extern-template/)

Abstract

Writing extern template statements is painful as we have to write out the template as it was originally declared, and not as it might be presented to the user.

Table of Contents

1 Revision History

1.1 Revision 0

2 Motivation

3 Wording

References

Informative References

§ 1. Revision History

§ 1.1. Revision 0

Initial Release 🎉

§ 2. Motivation

Writing `extern template` statements has become so painful that it is one of the least used features in C++ outside of some vendors and extremely large C++ projects such as LLVM. When performing a search across all of GitHub's source code, all instances of `extern template` were found to be in test repositories for developers to play around with to understand how they worked. Most have not been touched since the first C++11 compliant compilers were released.

To best display the primary issue at hand, let's look at what happens when we try to `extern template std::string`:

```
extern template class std::string;
```

Every compiler will reject the above, because we cannot `extern template` typedefs. Instead, we are required to know the actual definition of `std::string`, minus default template parameters. To actually `extern template` a class like `std::string`, one must write:

```
extern template class std::basic_string<char>;
```

This can be surprising to many since `extern template class std::vector<std::string>` works just fine.

Modifying the standard to let us `extern template` aliases is desirable.

Note: This paper does not attempt to permit specializations of alias templates, nor does it permit identifiers defined via a typedef. Instead an *alias-declaration* must be used. This is done to encourage the newer syntax when working with `extern template`.

§ 3. Wording

Wording is relative to [\[N4762\]](#)

In Section 12.8.2 Explicit instantiation [**temp.explicit**] paragraph 3, insert:

³ If the explicit instantiation is for a class or member class, **either** the *elaborated-type-specifier* in the *declaration* shall include a *simple-template-id* **, or the *declaratation* shall be a *template-name* that refers to an *alias template***; otherwise the *declaration* shall be a *simple-declaration* whose *init-declarator-list* comprises a single *init-declarator* that does not have an *initializer*. If the explicit instantiation is for a function or member function, the *unqualified-id* in the *declarator* shall be either a *template-id* or, where all template arguments can be deduced, a *template-name* or *operator-function-id*. [Note: The declaration may declare a *qualified-id*, in which case the *unqualified-id* of the *qualified-id* must be a *template-id*. —end note] If the explicit instantiation is for a member function, a member class or a static data member of a class template specialization, the name of the class template specialization in the *qualified-id* for the member name shall be a *simple-template-id*. If the explicit instantiation is for a variable template specialization, the *unqualified-id* in the *declarator* shall be a *simple-template-id*. An explicit instantiation shall appear in an enclosing namespace of its template. If the name declared in the explicit instantiation is an unqualified name, the explicit instantiation shall appear in the namespace where its template is declared or, if that namespace is inline (9.7.1), any namespace from its enclosing namespace set. [Note: Regarding qualified names in declarators, see 9.2.3. — end note]

§ References

§ Informative References

[N4762]

Richard Smith. [Working Draft, Standard for Programming Language C+](#). 7 July 2018. URL: <https://wg21.link/n4762>